

COMSYS 705 Assignment2 Report

Ruoyu Zhang rzha901

Results from SPIN Solvers for Q1

```
rzha901@DIP1R81Ubu203:~/Desktop$ ./pan -a -N P1_safety
pan: ltl formula P1_safety

(Spin Version 6.5.1 -- 20 December 2019)
+ Partial Order Reduction

Full statespace search for:
    never claim          + (P1_safety)
    assertion violations + (if within scope of claim)
    acceptance cycles   + (fairness disabled)
    invalid end states   - (disabled by never claim)

State-vector 36 byte, depth reached 57, errors: 0
    58 states, stored
    43 states, matched
    101 transitions (= stored+matched)
    0 atomic steps
hash conflicts:      0 (resolved)

Stats on memory usage (in Megabytes):
    0.004    equivalent memory usage for states (stored*(State-vector + overhead))
    0.288    actual memory usage for states
    128.000  memory used for hash table (-w24)
    0.534    memory used for DFS stack (-m10000)
    128.730  total actual memory usage

unreached in proctype P1
    Q1.pml:68, state 21, "-end-"
    (1 of 21 states)
unreached in claim P1_safety
    _spin_nvr.tmp:8, state 10, "-end-"
    (1 of 10 states)

pan: elapsed time 0 seconds
```

```
rzha901@DIP1R81Ubu203:~/Desktop$ ./pan -a -N P2_liveness
pan: ltl formula P2_liveness

(Spin Version 6.5.1 -- 20 December 2019)
+ Partial Order Reduction

Full statespace search for:
    never claim          + (P2_liveness)
    assertion violations + (if within scope of claim)
    acceptance cycles   + (fairness disabled)
    invalid end states   - (disabled by never claim)

State-vector 36 byte, depth reached 20, errors: 0
    38 states, stored (76 visited)
    55 states, matched
    131 transitions (= visited+matched)
    0 atomic steps
hash conflicts:      0 (resolved)

Stats on memory usage (in Megabytes):
    0.002    equivalent memory usage for states (stored*(State-vector + overhead))
    0.288    actual memory usage for states
    128.000  memory used for hash table (-w24)
    0.534    memory used for DFS stack (-m10000)
    128.730  total actual memory usage

unreached in proctype P1
    Q1.pml:61, state 16, "numProcesses = (numProcesses-1)"
    Q1.pml:66, state 17, "flag[_pid] = 0"
    Q1.pml:68, state 21, "-end-"
    (3 of 21 states)
unreached in claim P2_liveness
    _spin_nvr.tmp:15, state 6, "-end-"
    (1 of 6 states)

pan: elapsed time 0 seconds
```

```
rzha901@DIP1R81Ubu203:~/Desktop$ ./pan -a -N P3_liveness
pan: ltl formula P3_liveness

(Spin Version 6.5.1 -- 20 December 2019)
+ Partial Order Reduction

Full statespace search for:
    never claim          + (P3_liveness)
    assertion violations + (if within scope of claim)
    acceptance cycles   + (fairness disabled)
    invalid end states   - (disabled by never claim)

State-vector 36 byte, depth reached 57, errors: 0
    113 states, stored (168 visited)
    228 states, matched
    396 transitions (= visited+matched)
    0 atomic steps
hash conflicts:      0 (resolved)

Stats on memory usage (in Megabytes):
    0.007    equivalent memory usage for states (stored*(State-vector + overhead))
    0.288    actual memory usage for states
    128.000  memory used for hash table (-w24)
    0.534    memory used for DFS stack (-m10000)
    128.730  total actual memory usage

unreached in proctype P1
    Q1.pml:68, state 21, "-end-"
    (1 of 21 states)
unreached in claim P3_liveness
    _spin_nvr.tmp:26, state 13, "-end-"
    (1 of 13 states)

pan: elapsed time 0 seconds
```

The above picture shows the result of model checking Peterson's mutual exclusion algorithm using SPIN. My promela code is an algorithm for two processes ($N=2$), using a shared flag and turn variable to coordinate access to the shared critical section. Each individual process will wait until it is its turn before entering the critical section, ensuring that only one process can enter the critical section at a time.

The key components of the code are as follows:

Flags (flag[N]): Each process has a flag indicating its intention to enter the critical section.

Turn (turn): Indicates which process has priority in case of a conflict.

Steps; Steps array for Peterson's algorithm stages.

Critical Section: The code uses an atomic block to ensure that only one process can access the critical section at a time.

The properties verified are:

Safety Property (P1_safety): Ensures that no more than one process is in the critical section simultaneously.

Liveness Property (P2_liveness): Guarantees that if a process is waiting, it will eventually enter the critical section.

Liveness Property (P3_liveness): Ensures that any process not in the critical section will eventually get a chance to enter it.

Verification Results

We verified the following LTL properties using SPIN:

1. Safety Property: P1_safety

- **Property:** $\text{ltl P1_safety } \{ \square(\text{numProcesses} \leq 1) \}$
- **Result:** The verification showed that this property holds true, as no assertion violations were found. This indicates that Peterson's algorithm effectively prevents multiple processes from entering the critical section simultaneously, ensuring mutual exclusion.
- **Summary:** The state-vector size was 36 bytes, and 58 states were explored, with no errors detected. This confirms that mutual exclusion is maintained throughout the execution.

2. Liveness Property: P2_liveness

- **Property:** $\text{ltl } P2_liveness \{ \Diamond \text{waitProcess} \}$
- **Result:** The verification demonstrated that this property holds, indicating that if a process is waiting, it will eventually enter the critical section. No deadlock scenarios were detected.
- **Summary:** The state-vector size was 36 bytes, and 38 states were stored with 131 transitions explored. The absence of assertion violations confirms that Peterson's algorithm guarantees progress for each waiting process.

3. Liveness Property: P3_liveness

- **Property:** $\text{ltl } P3_liveness \{ [](!(\text{numProcesses} == 1) \rightarrow \Diamond (\text{numProcesses} == 1)) \}$
- **Result:** This property also held true during verification, showing that any process not currently in the critical section will eventually get a chance to enter it.
- **Summary:** The state-vector size was 36 bytes, with 113 states stored and 396 transitions explored. No errors were found, ensuring that the system is free from starvation.

Conclusion

As can be seen from the SPIN results in our screenshot, Peterson's mutual exclusion algorithm correctly enforces mutual exclusion and ensures liveness of the two processes. The properties of the code verified by spin show that multiple processes cannot enter the critical section at the same time, and any waiting process will eventually continue. Therefore, our completed code implementation in Q1 meets the required safety and liveness specifications.

Results from SMT Solvers for Q2

The Q2 goal is to use an SMT solver to determine if these two equations are logically equivalent for all possible combinations of Boolean values (A, B, C). If they are equivalent, they should produce the same output for any given input.

The boolean equation:

$$Y = (\neg A \wedge B \wedge C) \vee (A \wedge \neg B \wedge C) \vee (A \wedge B \wedge \neg C) \vee (A \wedge B \wedge C) - (1)$$

$$Y' = (A \wedge B) \vee (B \wedge C) \vee (A \wedge C) - (2)$$

The Python script used the following steps:

Define Boolean Variables: Three boolean variables A, B, and C were defined using Bool() from Z3.

Define the Equations: The two equations Y and Y' were defined using the logical operations And, Or, and Not from Z3.

Create Solver and Add Constraints: A solver instance was created, and the constraint $Y \neq Y'$ was added to check if there exists a scenario where the equations yield different outputs.

Check Equivalence: The solver was then used to check if the constraint could be satisfied. If it could, an example was provided to demonstrate the discrepancy between Y and Y'.

Results

After running the script in a Linux environment, the following output was obtained:

```
rzha901@D1P1RB1Ubu203:~/Desktop$ python3 Q2.py  
Equation 1 and Equation 2 are equivalent.
```

The SMT solver found that there is no combination of values for A, B, and C that makes the two equations differ in their output. Therefore, the solver concluded that the two equations are logically equivalent.

Conclusion

The SMT solver successfully verified that Equation 1 (Y) and Equation 2 (Y') are logically equivalent. This means that both equations produce the same output for all possible Boolean values of A, B, and C. This result confirms the correctness of both equations and shows that they can be used interchangeably.